

IFN647 Week 4 Workshop Tasks

Solutions

Task 1

Solution (discussion points):

There are lots of variant formulations and combinations. Whatever formulation is used, the unit-length-normalized TF*IDF scores are the precomputed and stored, so that similarity comparison is just a dot product.

Term Frequency (*tf*):

The term frequency *tf* in *tf*idf* can be the raw term frequency $f_{d,t}$ (the number of term *t*'s appearance in document *d*). However, a term that occurs 10 times is not generally 10 times as important as a term that occurs once. Therefore, an alternative formulation of the *tf* in a document *d* can be:

$$1 + \log(f_{t,d})$$

Inverse Document Frequency (*idf*):

If *N* is the number of documents in a given document collection *C* (or a dataset), and df_t is the number of documents that contain term *t*. Then the *idf* of term *t* in a collection *C* is defined as:

$$\text{idf}_t = \log \frac{N}{df_t}$$

where df_t is equivalent to n_t .

For example, suppose *C* includes 10 documents, and a word "tutorial" appears in three documents. Then, mathematically, its Inverse-Document Frequency, $\text{idf}_t = \log(10/3)$.

Smoothing and Document-Length-Normalized version:

$$\text{tfidf}(t, d) = \frac{(1 + \log(f_{t,d})) \cdot \log \frac{N}{df_t}}{\sqrt{\sum_{i=1}^T \left[(1 + \log(f_{i,d})) \cdot \log \frac{N}{df_i} \right]^2}}$$

where $N = |C|$, and T is the total number of terms in collection C . To prevent longer documents from receiving systematically larger weights, **normalization** is applied in this equation.

Task 2

Solution

	Term1	Term2	Term3	Term4	Term5
D1	3	0	0	5	7
D2	5	3	4	6	0
D3	0	0	5	4	6
D4	9	0	0	1	2
D5	0	1	0	3	2
D6	3	0	2	4	4
<i>df</i>	4	2	3	6	5

Task 2.3

Solution

```
#define a function to calculate df
def c_df(docs):
    """Calculate DF of each term in vocab and return as term:df dictionary."""
    df_ = {}
    for id, doc in docs.items():
        for term in doc.keys():
            try:
                df_[term] += 1
            except KeyError:
                df_[term] = 1
    return df_

# the document collection is represented as a dict
docs = {'D1':{'term1':3, 'term4':5, 'term5':7}, 'D2':{'term1':5, 'term2':3, 'term3':4, 'term4':6},
        'D3':{'term3':5, 'term4':4, 'term5':6}, 'D4':{'term1':9, 'term4':1, 'term5':2},
        'D5':{'term2':1, 'term4':3, 'term5':2}, 'D6':{'term1':3, 'term3':2, 'term4':4, 'term5':4}}

# test the input
print(docs)
print('\n')
# call the function
x = c_df(docs)
print('Display terms df from term 1 to term 5: \n')
y = {k: v for k, v in sorted(x.items(), key=lambda item: item[0])}
print(y)
```

Task 4

Solution

	Relevant	Non-relevant	Total
$d_1 = 1$	$r_i = 3$	$n_i - r_i = 1$	$n_i = 4$
$d_1 = 0$	$R - r_i = 0$	$(N - R) - (n_i - r_i) = N - n_i - R + r_i = 3$	$N - n_i = 3$
Total	$R = 3$	$N - R = 4$	$N = 7$

	Relevant	Non-relevant	Total
$d_2 = 1$	$r_i = 0$	$n_i - r_i = 4$	$n_i = 4$
$d_2 = 0$	$R - r_i = 3$	$N - n_i - R + r_i = 0$	$N - n_i = 3$
Total	$R = 3$	$N - R = 4$	$N = 7$

	Relevant	Non-relevant	Total
$d_3 = 1$	$r_i = 2$	$n_i - r_i = 1$	$n_i = 3$
$d_3 = 0$	$R - r_i = 1$	$N - n_i - R + r_i = 3$	$N - n_i = 4$
Total	$R = 3$	$N - R = 4$	$N = 7$